

ZIMON

Zebrafish Integrated Motion & Optical Neuroanalysis Chamber

Complete Project Documentation

Version 2.0.0 | Desktop Application (Windows 10/11 64-bit)

Document covers:	Installation · Features · Hardware · File Structure · Installer
Hardware support:	Arduino · FLIR Camera · Basler Camera · USB Webcam
Tech stack:	Python 3.12 · PyQt6 · OpenCV · SQLite · PyInstaller
Prepared:	May 2026

ZIMON - Complete Project Documentation

Zebrafish Integrated Motion & Optical Neuroanalysis Chamber

Version 2.0.0 | Desktop Application (Windows 10/11 64-bit)

Table of Contents

- * [Project Overview](#1-project-overview)
- * [System Requirements](#2-system-requirements)
- * [Installation & Setup](#3-installation--setup)
- * [Running the Application](#4-running-the-application)
- * [Screen-by-Screen Feature Guide](#5-screen-by-screen-feature-guide)
- * [PPT Requirements Mapping](#6-ppt-requirements-mapping)
- * [Hardware Connections](#7-hardware-connections)
- * [Database & Data Storage](#8-database--data-storage)
- * [Project File Structure](#9-project-file-structure)
- * [Building the Installer (.exe)](#10-building-the-installer-exe)
- * [Known Limitations](#11-known-limitations)
- * [Troubleshooting](#12-troubleshooting)

1. Project Overview

ZIMON is a laboratory desktop application for zebrafish behavioral neuroscience research. It enables researchers to:

- * Control camera systems (FLIR thermal, Basler high-speed, standard webcams)
- * Deliver precisely-timed stimuli to zebrafish (light, vibration, sound, water flow)
- * Build and run multi-step experimental protocols
- * Record video and synchronize stimulus events with timing data
- * Log and replay experiments, export data in multiple formats
- * Manage lab users with role-based access control

Development History

The client provided two legacy code archives:

- * Behavior-system-code-26-main.zip - Version 1 baseline (tabbed PyQt6 UI)
- * ZimonAll4Versions-master.zip - Version 2 with modular architecture

These were unified into a single codebase at ZIMON_App/ using V2 as the base, with all client-requested features

built on top.

2. System Requirements

Software

Requirement	Minimum	Recommended
OS	Windows 10 64-bit	Windows 11 64-bit
Python	3.8	3.12
RAM	4 GB	8 GB
Disk	500 MB	2 GB (for recordings)
Display	1280×720	1920×1080

Python Packages (auto-installed)

```
PyQt6 >= 6.4.0
numpy >= 1.21.0
opencv-python >= 4.5.0
pyserial >= 3.5
Pillow >= 8.0.0
scipy >= 1.7.0
pandas >= 1.3.0
matplotlib >= 3.5.0
```

Optional Hardware SDKs

Hardware	SDK Required	Install Method
Basler cameras	Pylon SDK + pypylon	`pip install pypylon` + Pylon installer
FLIR thermal cameras	Spinnaker SDK + PySpin	Manual from flir.com
Arduino controller	None (uses pyserial)	Already included
USB webcam	None (uses OpenCV)	Already included

3. Installation & Setup

Method A - Run from Source (Development)

Step 1: Install Python 3.12

Download from <https://python.org> - check "Add to PATH" during install.

Step 2: Install dependencies

```
cd C:\Users\Bluecloud\Desktop\Projects\Zimon\ZIMON_App
pip install PyQt6 numpy opencv-python pyserial Pillow scipy pandas matplotlib
```

Step 3: (Optional) Install Basler camera support

```
pip install pypylon
```

Then install Basler Pylon SDK from <https://www.baslerweb.com/en/downloads/software-downloads/>

Step 4: (Optional) Install FLIR camera support

- * Download Spinnaker SDK from <https://www.flir.com/products/spinnaker-sdk/>
- * Run installer as Administrator
- * During install, select Custom and tick Python Bindings
- * Restart the application - FLIR cameras will appear automatically

Step 5: Flash Arduino firmware

- * Open arduino/zfish_controller.ino in Arduino IDE
- * Connect Arduino Mega via USB
- * Select correct COM port and board type
- * Upload firmware

Method B - Run from Installer (.exe)

Use the pre-built installer in dist/ZIMON/ZIMON.exe.

No Python installation required - all dependencies are bundled.

```
dist/
  ZIMON/
    ZIMON.exe      <- Double-click to launch
    (all supporting files automatically included)
```

4. Running the Application

From Source

```
cd C:\Users\Bluecloud\Desktop\Projects\Zimon\ZIMON_App
python main_v2.py
```

From Installer

Double-click dist/ZIMON/ZIMON.exe

First Launch

On first launch the app automatically creates a local SQLite database at:

```
C:\Users\<username>\.zimon\zimon.db
```

A default admin account is seeded:

- * Username: admin
- * Email: admin@zimon.lab
- * Password: zimon2024

[!] [!] Change the admin password immediately in production use.

5. Screen-by-Screen Feature Guide

Screen 1 - Login

The entry point of the application. Two-panel layout: dark blue brand panel on the left, white login form on the right.

Features:

Feature	Description
Email or Username field	Accept either the account email address or username
Password field	Masked input
Remember me	Saves username preference for next session
Forgot Password?	Displays message to contact lab administrator

Login button	Authenticates against local database (bcrypt-equivalent SHA-256)
No self-registration	By client requirement - only admins can create new accounts
Bottom status bar	Shows live Camera / Chamber / Temperature hardware status

Access Levels:

- * Admin - Full access including User Management
- * Researcher - Full experimental access, no user management
- * Student - Full experimental access, no user management

Screen 2 - Environment

Hardware status dashboard. Accessible via the Environment tab in the top navigation.

Camera Devices Section:

Feature	Description
Machine Vision (Larval)	Shows FLIR/Basler camera connection status
USB Camera (Adult Top)	Second camera status
USB Camera (Adult Side)	Third camera status
Connected / Disconnected badges	Live status, updates every 2 seconds
Test button per camera	Starts a preview stream for that camera
System Ready indicator	Green when 1 camera connected

Stimulus Devices Section:

Feature	Description
Light	Tests IR/White light via Arduino `ENV_IR 50` command
Vibration	Tests vibration motor via `VIBRATE_TIMED 500` (500ms burst)

Buzzer	Tests buzzer via `BUZZER_ON` -> 500ms -> `BUZZER_OFF`
Water Flow	Tests pump via `PUMP_ON` -> 300ms -> `PUMP_OFF`
Test buttons	Each fires the actual Arduino hardware command
System Ready: YES	Confirms all stimulus devices responding

Screen 3 - Adult Mode (Stimulus Control)

The main experimental screen for adult zebrafish. Accessible via the Adult tab.

Three-column layout: Stimulus Control | Live Video + Timeline | Assay Select.

Left Panel - Stimulus Control:

Feature	Description
IR / White / RGB buttons	Select light type (exclusive toggle)
Intensity slider	0-100%, displayed as percentage
Continuous / Pulse mode	Radio button toggle
Frequency slider	0-50 Hz
Pulse Width slider	Pulse duration in milliseconds
Duration slider	Total stimulus duration in seconds
Buzzer - Tone / Noise / File	Audio type selector
Amplitude	Volume/strength level
Duration	Buzzer duration

All stimulus controls fire Arduino hardware commands in real-time when the experiment is running.

Center Panel - Video + Timeline:

Feature	Description
Live video feed	Connected to active camera (FLIR/Basler/webcam) at up to 60 FPS display
> Start button	Begins experiment timer and activates stimulus hardware

Stop button	Halts all stimuli immediately
Duration display	Live elapsed time (MM:SS format)
Manual / Protocol mode	Manual = immediate control; Protocol = run saved protocol steps
FPS selector	Display frame rate for the video panel
Timeline visualization	Horizontal color-coded bars showing Baseline / Light Pulse / Recovery phases
Protocol dropdown	Select a saved protocol to run

Right Panel - Assay Select:

Feature	Description
TOP / SIDE radio buttons	Select camera angle for current assay
System Ready: YES	Real-time hardware readiness check
Startle Response	Preset assay configuration
Light/Dark Test	Preset assay configuration
Load Assay	Opens a saved assay
Create New -> Protocol Builder	Navigates to Protocol Builder screen
Run Protocol button	Executes the selected protocol on hardware

Screen 4 - Protocol Builder

Build, save, and run multi-step experimental protocols. Accessible via the Protocol Builder tab.

Left Panel - Protocol Editor:

Feature	Description
Protocol Name	Text field for naming the protocol
Description	Optional protocol description

+ Baseline button	Inserts a baseline stage at the start
Light / Buzzer / Vibration / Water Flow buttons	Add a step of that type
Step list	Ordered list of all steps in the protocol
Edit button per step	Opens step editor dialog with all parameters
Delete () button per step	Removes that step
Timeline visualization	Proportional color bars showing each step duration

Step Editor Dialog (per step type):

Step Type	Parameters
Baseline	Duration (seconds)
Light	Flash Type (IR/White/RGB), Intensity %, Pulse Width (ms), Duration (ms)
Buzzer	Tone type (Tone/Noise/File), Amplitude, Duration (ms)
Vibration	Frequency (Hz), Duration (ms)
Water Flow	Duration (ms)

Right Panel - Protocol Summary:

Feature	Description
Protocol Summary list	Step-by-step breakdown with durations
Total Runtime	Auto-calculated total duration
Load Protocol dropdown	Load any previously saved protocol
Load button	Loads selected protocol into editor
Save Protocol button	Saves to local SQLite database
Test Run button	Shows dry-run summary (no hardware activation)
> Run Protocol button	Executes all steps sequentially on connected Arduino

Stop button	Halts execution mid-run
Status label	Shows current step being executed

When Run Protocol is clicked:

- * Creates an experiment record in the database
- * Executes each step with correct timing
- * Logs timestamped events for every stimulus (on/off times, values)
- * On completion or stop, saves the complete events log to the database

Screen 5 - Experiments

View, replay, and export past experiments. Accessible via the Experiments tab.

Left Panel - Experiments Log:

Feature	Description
Period filter dropdown	Last 7 Days / Last 30 Days / Last 90 Days / All Time
Protocol filter dropdown	Filter by specific protocol name
Search box	Search by experiment name or ID
Experiment list	Date · Name · Status (Completed/Failed/Running) badges
Pagination	20 items per page with navigation

Center Panel - Experiment Playback:

Feature	Description
Video area	Shows recorded experiment footage (opens in system player)
> / / >> / controls	Playback transport buttons
Elapsed time display	MM:SS / total duration
Timeline tab	Visual Baseline / Stimulus / Recovery bars per stimulus track
Summary tab	Text summary of experiment parameters

Right Panel - Experiment Details:

Feature	Description
Experiment ID	Unique identifier (e.g. EXP_1746_0403)
Date / Time	When the experiment was run
Protocol	Protocol used
Duration	Total experiment duration
Camera	Camera used
FPS	Recording frame rate
Storage Path	Where the video file is saved
Open Folder button	Opens the storage folder in Windows Explorer
Copy Path button	Copies the storage path to clipboard
[replay] Replay button	Opens the experiment video
Export...	Opens export dialog
Export CSV	Opens export dialog (pre-selects CSV)
Export All (ZIP)	Opens export dialog (pre-selects all + ZIP)

Screen 6 - Export Dialog

Modal dialog for exporting experiment data in multiple formats.

Feature	Description
Raw Video (.mp4) checkbox	Copies the experiment video file
Events Log (.csv) checkbox	Exports timestamped stimulus events as CSV
Experiment Metadata (.json) checkbox	Exports all experiment fields as JSON
Protocol File (.json) checkbox	Exports the protocol steps as JSON
All Files as ZIP Archive checkbox	Bundles all selected files into a single .zip
Output path field	Destination folder (type or browse)

Browse button	Opens folder picker dialog
Cancel button	Dismisses dialog, no files written
Export button	Writes all selected files to the destination folder

Screen 7 - Larval Mode

Identical stimulus control to Adult mode, but with a Well Plate ROI overlay on the video for larval zebrafish experiments (multi-well plate tracking). Accessible via the Larval tab.

Feature	Description
Plate format dropdown	6-well / 12-well / 24-well / 48-well / 96-well
Well plate grid overlay	Click individual wells to select/deselect ROIs
Select All / Clear buttons	Bulk select or deselect all wells
Assay Select (TOP/SIDE)	Camera angle selection
Escape Response / Light Dark Test	Preset assay configurations
Create New -> Protocol Builder	Navigation link
Stimulus control panel	Identical to Adult mode

Screen 8 - Settings

Application configuration. Accessible via Protocol Builder -> Settings or the nav user menu.

Section	Feature	Description
Hardware Connection	Arduino Port dropdown	Lists all detected COM ports
	Refresh button	Re-scans available serial ports
	Connect button	Connects to selected port (fires PING handshake)

	Connection status dot	Green = connected, Red = not connected
	Baud Rate	9600 / 57600 / 115200
Camera Defaults	Default FPS	1-240 FPS
	Default Resolution	Width × Height in pixels
	Auto-start camera	Opens camera on app launch
Recording	Output folder	Default save location for recordings
	Filename prefix	Prefix for auto-named recording files
Application	Theme	Light / Dark (placeholder)
	Auto-save protocols	Saves protocol changes automatically
	Save Settings button	Applies all settings and connects Arduino

User Management (Admin only)

Accessible via top-right user chip -> Manage Users (admin accounts only).

Feature	Description
User list table	Shows all users with ID, username, email, role
+ Create User button	Opens form to create new account
Edit button per user	Opens form to change email, role, password
Deactivate button	Soft-deletes the user (cannot log in, record kept)
Roles	admin / researcher / student

6. PPT Requirements Mapping

The following table maps every screen and feature from the client's PowerPoint mockup to its implementation status.

PPT Screen	PPT Feature	Implementation	Status
Login	Email/password form	<code>`gui_v2/login_window.py`</code>	[YES] Done
Login	Forgot Password	Shows admin contact dialog	[YES] Done
Login	No self-registration	Admin-only user creation enforced	[YES] Done (client note Slide 6)
Login	Camera/Chamber/Temperature status bar	<code>`gui_v2/bottom_bar.py`</code>	[YES] Done
Environment	Camera device cards with Connected badges	<code>`EnvironmentPage._DeviceCard`</code>	[YES] Done
Environment	Test button per device	Fires Arduino command + preview	[YES] Done
Environment	Stimulus device cards (Light/Vib/Buzzer/Water)	<code>`_StimulusCard`</code> widgets	[YES] Done
Environment	System Ready indicators	Live polling every 2 seconds	[YES] Done
Adult	Stimulus Control panel (left)	<code>`StimulusControlPanel`</code>	[YES] Done
Adult	IR/White/RGB light type selection	Wired to <code>`arduino.set_ir_intensity()`</code>	[YES] Done
Adult	Intensity/Frequency/Pulse sliders	All sliders wired	[YES] Done
Adult	Live camera video (center)	<code>`VideoPanel`</code> + <code>`frame_ready`</code> signal	[YES] Done

Adult	Start/Stop experiment	Timer + hardware control	[YES] Done
Adult	Timeline (Baseline/Light Pulse/Recovery)	`TimelineBar` custom painter	[YES] Done
Adult	Assay Select TOP/SIDE	Radio buttons	[YES] Done
Adult	Startle Response / Light Dark Test	Preset assay rows	[YES] Done
Adult	Create New -> Protocol Builder	`navigate_to` signal -> page switch	[YES] Done
Protocol Builder	Protocol name + description	Text fields	[YES] Done
Protocol Builder	Add Steps (Light/Buzzer/Vibration/Water Flow)	Step type buttons + editor dialog	[YES] Done
Protocol Builder	Automated Protocol Timeline (visual bars)	`TimelineWidget` custom painter	[YES] Done
Protocol Builder	Protocol Summary sidebar	Step list + total runtime	[YES] Done
Protocol Builder	Save Protocol	SQLite via `db.save_protocol()`	[YES] Done
Protocol Builder	Load Protocol	SQLite via `db.list_protocols()`	[YES] Done
Protocol Builder	Test Run	Dry-run info dialog	[YES] Done
Protocol Builder	Run Protocol on hardware	Arduino step execution + events log	[YES] Done
Experiments	Experiments log list (left panel)	`ExperimentsLogPanel` from DB	[YES] Done
Experiments	Date/Protocol filter + search	Client-side filtering	[YES] Done

Experiments	Experiment Playback (video)	Opens recorded .mp4	[!] Opens system player (no in-app scrub)
Experiments	Timeline (Baseline/Stimulus/Recovery)	`PlaybackTimelineWidget`	[YES] Done
Experiments	Experiment Details panel	All 8 fields from DB	[YES] Done
Experiments	Open Folder / Copy Path	OS Explorer + clipboard	[YES] Done
Experiments	Export dialog with checkboxes	`ExportDialog`	[YES] Done
Experiments	Export writes actual files	`shutil`, `csv`, `json`, `zipfile`	[YES] Done
Export Dialog	Raw Video / Events / Metadata / Protocol / ZIP	All 5 checkboxes wired	[YES] Done

Client special requirement (PPT Slide 6):

[!] "Register from Login Screen - New Registration should be blocked. Only Admin should be allowed to create login for students with rights."

[YES] Implemented exactly as specified. The login screen has no registration link. New users can only be created through Admin -> Manage Users.

7. Hardware Connections

Arduino Controller

The Arduino runs custom firmware (arduino/zfish_controller.ino) and communicates over USB serial.

Setup:

- * Flash arduino/zfish_controller.ino to an Arduino Mega
- * Connect via USB
- * Open ZIMON -> Settings -> Hardware Connection
- * Select the correct COM port from the dropdown
- * Click Connect - the status dot turns green when successful

Auto-detection: The app can auto-detect the Arduino by scanning all COM ports and sending a PING command. The Arduino responds with ZIMON_OK.

Supported Commands:

Command	Function
`ENV_IR <0-100>`	Set IR light intensity (DAC, not PWM)
`ENV_WHITE <0-100>`	Set white light intensity
`RGB_SET <r> <g> <b>`	Set RGB LED colour (0-255 per channel)
`VIBRATE_TIMED <ms>`	Vibrate for specified milliseconds
`VIBRATE_ON / OFF`	Vibration motor toggle
`BUZZER_ON / OFF`	Buzzer toggle
`PUMP_ON / OFF`	Water flow pump toggle
`TEMP?`	Read temperature sensor (returns float degC)
`PING`	Connectivity check (responds `ZIMON_OK`)

Camera Support

USB Webcam (standard)

- * No setup required - OpenCV detects automatically
- * Appears as Webcam_0, Webcam_1, etc.
- * Resolution: up to 1920×1080 @ 30 FPS

Basler Scientific Camera

```
pip install pypylon
```

Then install Basler Pylon SDK

- * Appears automatically in camera dropdown after SDK install
- * Supports: resolution, FPS, exposure, gain control
- * Recommended settings: Adult 1440×1080 @ 120 FPS, Larval 1024×1024 @ 180 FPS

FLIR Thermal Camera

- * Download Spinnaker SDK from [flir.com](https://www.flir.com/products/spinnaker-sdk/)
- * Run as Administrator, select Custom -> Python Bindings
- * Verified compatible model: FLIR CM3-U3-13Y3M-CS (USB3)

- * After install, restart ZIMON - FLIR camera appears in dropdown

Camera Priority (auto-select order): FLIR -> Basler -> Webcam

8. Database & Data Storage

Location

```
C:\Users\<username>\.zimon\zimon.db (SQLite)
```

Tables

users

```
id | username | email | password (hashed) | role | created | active
```

protocols

```
id | name | description | steps (JSON array) | created_by | created | updated
```

experiments

```
id | exp_id | name | protocol_id | protocol_name | status  
  | mode | camera | duration_sec | storage_path  
  | events_log (JSON array) | created_by | started | finished
```

Events Log Format

Each run produces a timestamped events log stored as JSON:

```
[  
  {"time_sec": 0.0, "stimulus": "baseline", "action": "start", "value": "Baseline Stage"},  
  {"time_sec": 10.0, "stimulus": "light", "action": "on", "value": "IR 80%"},  
  {"time_sec": 10.1, "stimulus": "light", "action": "off", "value": ""},  
  {"time_sec": 10.1, "stimulus": "vibration", "action": "on", "value": "500ms"},  
  {"time_sec": 10.6, "stimulus": "vibration", "action": "off", "value": ""}  
]
```

Video Recordings

Recordings are saved as .mp4 (OpenCV mp4v codec) to:

C:\Users\<username>\Videos\ZIMON\<filename>_YYYYMMDD_HHMMSS.mp4

The path is configurable in Settings -> Recording -> Output Folder.

9. Project File Structure

```
ZIMON_App/

main_v2.py          <- Entry point (login -> main window loop)
version.py          <- Version string
requirements.txt     <- pip dependencies
zimon.spec          <- PyInstaller build spec
build_installer.bat  <- One-click build script

db/
  __init__.py
  database.py        <- SQLite: users, protocols, experiments

gui_v2/              <- All UI components
  login_window.py    <- Login screen
  main_window.py     <- Main window (top nav + stacked pages)
  nav_bar.py         <- Top navigation bar
  bottom_bar.py      <- Bottom status bar
  hardware_bridge.py <- Central hardware API (camera + arduino)
  video_panel.py     <- Live camera feed widget
  frame_utils.py     <- numpy -> QImage conversion
  tracking_overlay.py <- Tracking visualization overlay
  user_management.py <- Admin user CRUD dialog
  styles_v2.qss      <- Global stylesheet (light purple/white theme)

pages/
  __init__.py
  adult_page.py      <- Adult stimulus control + video (Screen 3)
  larval_page.py     <- Larval Well/ROI view (Screen 3 - Larval)
  environment_page.py <- Hardware status dashboard (Screen 2)
  protocol_builder_page.py <- Protocol editor (Screen 4)
  experiments_page.py <- Experiments log + export (Screen 5/6)
  settings_page.py   <- App settings (Screen 8)
  recording_page.py  <- Recording controls
  stimulus_page.py   <- Stimulus quick-control
  camera_settings_page.py <- Camera parameter settings
  multi_angle_page.py <- Multi-camera setup
  well_roi_page.py   <- Well ROI configuration

backend/
  arduino_controller.py <- Arduino serial (PING, all commands)
  camera_interface.py  <- Multi-camera (FLIR/Basler/webcam)
  camera_manager.py    <- High-level camera API with Qt signals
  flir_camera.py       <- FLIR Spinnaker SDK integration
  recording_manager.py  <- Non-blocking MP4 recording
```

frame_relay.py	<- Thread-safe frame queuing
mode_profiles.py	<- Adult/Larval camera presets
experiment_runner.py	<- Legacy experiment execution engine
zebrazoom_integration.py	<- ZebraZoom analysis wrapper
analysis/	
metrics.py	<- Behavioral metrics calculation
plots.py	<- Plot generation (matplotlib)
tracking/	
tracker.py	<- Kalman filter position tracking
detector.py	<- Object detection
background.py	<- Background subtraction
preprocessing.py	<- Image preprocessing
exporter.py	<- Result export
arduino/	
zfish_controller.ino	<- Arduino firmware (flash this to the board)
config/	
hardware_config.json	<- Hardware configuration defaults
dist/	
ZIMON/	
ZIMON.exe	<- Built installer (run build_installer.bat to create)

10. Building the Installer (.exe)

Quick build

```
cd C:\Users\Bluecloud\Desktop\Projects\Zimon\ZIMON_App
build_installer.bat
```

Manual build

```
pyinstaller zimon.spec --noconfirm
```

Output

```
dist/ZIMON/ZIMON.exe    <- 4.6 MB launcher
dist/ZIMON/             <- ~220 MB complete folder (all dependencies bundled)
```

Distributing to the client

Option A - Folder delivery:

Zip the entire dist/ZIMON/ folder and send it. Client unzips and runs ZIMON.exe.

Option B - Proper installer (recommended):

Use Inno Setup (free) to wrap dist/ZIMON/ into a single ZIMON_Setup.exe with:

- * Start Menu shortcut
- * Desktop shortcut
- * Uninstaller
- * Installation wizard

Basic Inno Setup script template is easy to generate - just point it at the dist/ZIMON/ folder.

Rebuild after code changes

```
build_installer.bat
```

This cleans previous builds and rebuilds from scratch automatically.

11. Known Limitations

Item	Detail	Workaround
In-app video scrubbing	The Experiments playback panel shows the video area but does not have a built-in frame-scrubbing player	Click **Replay** or **Open Folder** - video opens in the system's default video player (VLC, Windows Media Player, etc.)
Remember Me persistence	Saves username to UI but does not create a persistent OS-level session token	User must re-enter password after closing the app (by design for lab security)
FLIR SDK	PySpin is not installable via pip	Must be installed manually from the FLIR website as described in Section 7
pypylon on fresh machines	pip install pypylon alone is not sufficient	Basler Pylon SDK installer must also be run
Larval mode stimulus	Stimulus controls are wired to UI but the Larval page's hardware bridge connection is the same as Adult	No separate Larval-specific stimulus profile - both use the same Arduino commands

12. Troubleshooting

App won't start

```
pip install PyQt6 numpy opencv-python pyserial
```

Then try again.

"pypylon not available - Basler cameras disabled"

This is an expected warning, not an error. The app works fine with webcam. Install Pylon SDK + pypylon to enable Basler cameras.

"FLIR Spinnaker SDK not available"

Expected warning. Install Spinnaker SDK from FLIR website to enable FLIR cameras.

Arduino not connecting

- * Check device manager for the COM port number
- * Settings -> Hardware Connection -> select the correct port -> Connect
- * If still failing: ensure arduino/zfish_controller.ino is flashed to the board
- * Try a different baud rate (115200 is default)

Camera not appearing in dropdown

- * Environment page -> Test button for the camera
- * If no cameras found, Settings -> Camera Defaults -> uncheck Auto-start, restart app
- * For Basler: ensure Pylon SDK and pypylon are both installed

Database errors on first launch

The database is auto-created at ~/.zimon/zimon.db. If this fails:

```
mkdir %USERPROFILE%\zimon
```

Rebuild installer fails

```
pip install pyinstaller  
pyinstaller zimon.spec --noconfirm --clean
```

Document last updated: May 2026

ZIMON v2.0.0 | Developed for zebrafish behavioral neuroscience laboratory